

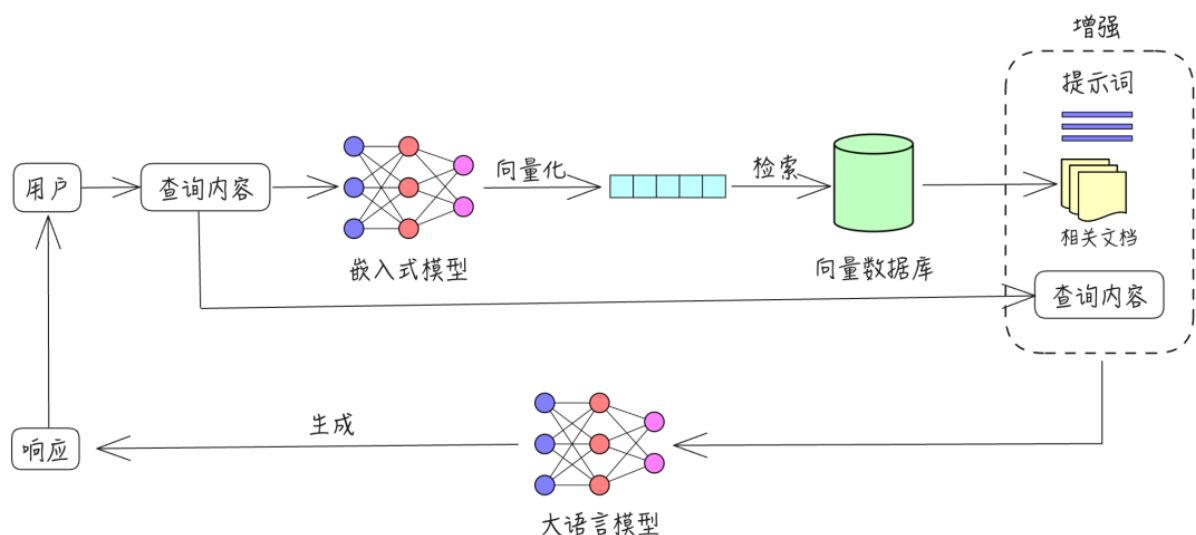
RAG(检索增强生成)简介：

检索增强生成（RAG）是一种优化大型语言模型（LLM）输出的方法，使其能够在生成响应之前引用训练数据之外的权威知识库。LLM 使用海量数据进行训练，拥有数十亿个参数，能够执行诸如回答问题、翻译语言和完成句子等任务。RAG 在 LLM 强大功能的基础上，通过访问特定领域或组织的内部知识库，而无需重新训练模型，进一步提升了其输出的相关性、准确性和实用性。这是一种经济高效的改进方法，适用于各种情境。

RAG 包含三个主要过程：检索、增强和生成。

- **检索**：根据用户的查询内容，从外部知识库获取相关信息。具体而言，将用户的查询通过嵌入模型转换为向量，以便与向量数据库中存储的相关知识进行比对。通过相似性搜索，找出与查询最匹配的前 K 个数据。
- **增强**：将用户的查询内容和检索到的相关知识一起嵌入到一个预设的提示词模板中。
- **生成**：将经过检索增强的提示词内容输入到大型语言模型中，以生成所需的输出。

以下是流程图：



RAG 解决的 LLM 应用痛点：

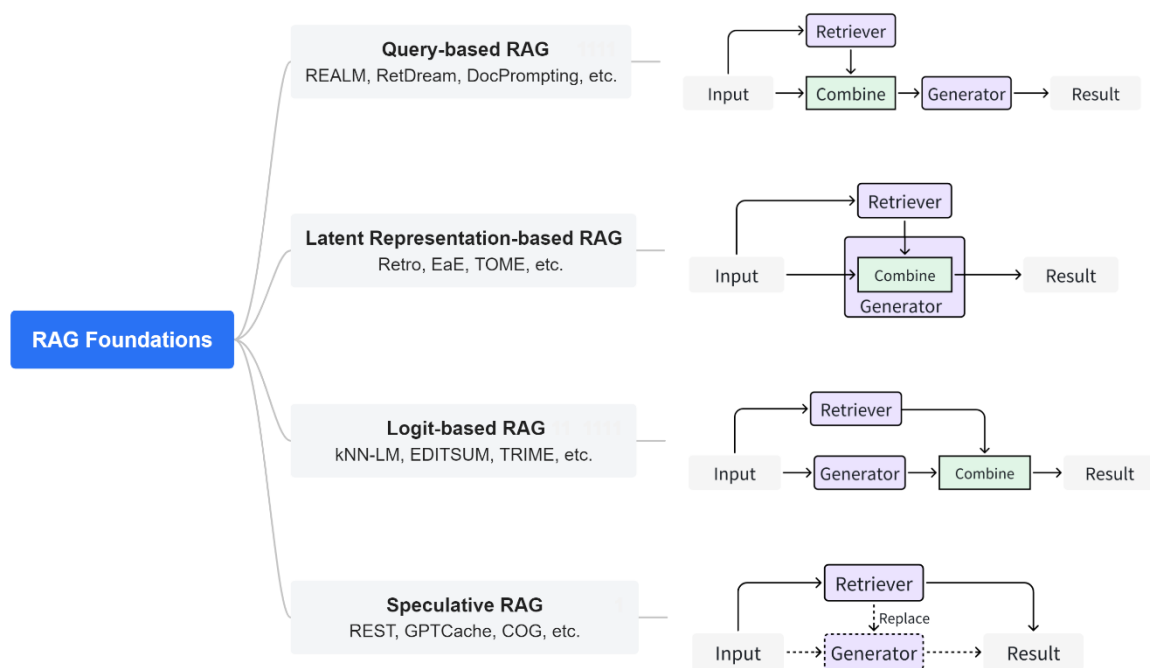
大型语言模型（LLM）在应用中面临一些已知挑战，包括：

- 在没有答案的情况下提供虚假信息。
- 当用户需要特定的当前响应时，提供过时或通用的信息。
- 从非权威来源创建响应。
- 由于术语混淆，不同的培训来源使用相同的术语来谈论不同的事情，从而产生不准确的响应。

RAG（检索增强生成）旨在缓解甚至解决以下大模型落地应用的痛点：

- **垂直领域知识的幻觉**：通过检索外部权威知识库，RAG 可以提供更准确和可靠的领域特定知识，减少生成幻觉的可能性。
- **大模型知识持续更新的困难**：无需重新训练模型，RAG 可以通过访问最新的外部知识库，保持输出的时效性和准确性。
- **无法整合长尾语义知识**：RAG 能够从广泛的知识库中检索长尾语义知识，从而生成更丰富和全面的响应。
- **可能泄露的训练数据隐私问题**：通过使用外部知识库而不是依赖内部训练数据，RAG 减少了隐私泄露的风险。
- **支持更长的上下文**：RAG 可以通过检索相关信息，提供更长和更详细的上下文支持，从而提高响应的质量和连贯性。

RAG 工作类型：

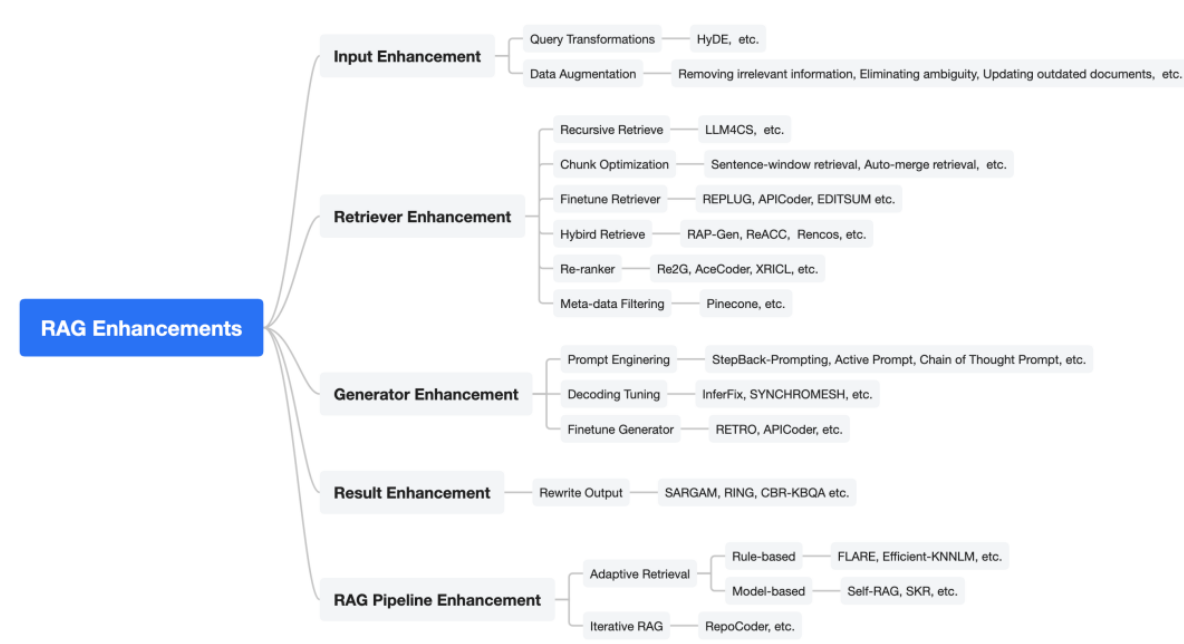


1. **Query-based RAG**：基于查询的 RAG 也称为提示增强。它将用户的查询与从文件中检索到的信息直接整合到语言模型输入的初始阶段。这种模式是 RAG 应用中广泛采用的方法。一旦检索到文档，它们的内容就会与用户的原始查询合并，创建一个组合输入序列。这个增强序列随后被输入到预先训练好的语言模型中，以生成回复。
2. **Latent Representation-based RAG**：在基于隐式表示的 RAG 框架中，检索到的对象作为隐式表示融入生成模型，从而提高模型的理解能力和生成内容的质量。在这种框架中，生成模型与检索对象的潜在表征相互作用，提高了生成内容的准确性。这种方

法在处理代码、结构化知识和多模态数据方面显示出巨大的潜力和适应性。特别是在代码相关的领域，如 EDITSUM、BASHEXPLAINER 和 RetrieveNEdit 等技术，采用 FiD 方法，通过编码器处理的融合来促进整合。Re2Com 和 RACE 等方法也采用了为不同类型输入设计多个编码器的设计。

3. **Logit-based RAG**: 在基于对数似然的 RAG 中，生成模型在解码过程中通过对数融合检索信息。通常，对数通过模型求和或组合，以产生逐步生成的概率。在代码到文本转换任务中，Rencos 并行生成检索代码的多个摘要候选，然后使用编辑距离进行规范化，计算最终概率以选择最匹配原始代码的摘要输出。在代码摘要任务中，EDITSUM 通过在概率级别整合原型摘要来提高摘要生成质量。对于文本到代码任务，kNN-TRANX 模型结合信心网络和元知识来合并检索到的代码片段。它利用 seq2tree 结构生成与输入查询紧密匹配的目标代码，提高代码生成的准确性和相关性。这种方法特别适合序列生成任务。它侧重于生成器的训练，并且可以设计出新颖的方法，有效地利用获取的概率分布，以适应后续任务。
4. **Speculative RAG**: 推测式 RAG (Speculative RAG) 旨在通过利用检索而非纯生成来节省资源并加快响应速度。REST 技术通过用检索替代推测解码中的小型模型，实现了草稿生成。GPTCache 通过构建语义缓存来存储大型语言模型 (LLM) 的响应，解决了使用 LLM API 时的高延迟问题。推测式 RAG 目前主要适用于序列数据。它解耦了生成器和检索器，使得预训练模型可以直接作为组件使用。在这个范式下，可以探索更广泛的策略，有效利用检索到的内容。

RAG 优化方案:



1. 输入增强 (Input Enhancement)

输入是指用户的查询，它最初被馈送到检索器中。输入的质量显著影响检索阶段的最终结果，

因此增强输入变得至关重要。输入增强主要包括查询转换和数据增强两种方法。

1. 查询转换 (Query Transformation)

查询转换通过修改输入查询来增强检索结果。Query2doc 和 HyDE 首先使用原始查询生成伪文档，然后使用该伪文档作为检索的查询。伪文档包含更丰富的相关信息，有助于获取更精确的结果。Query2doc 和 HyDE 通过这种方法提升了检索的准确性。TOC 则通过递归的检索增强澄清，针对模糊问题构建树结构的去歧义问题，从而生成这些模糊问题的全面答案。在构建树结构的过程中，TOC 使用了自我验证的剪枝方法，确保每个节点的事实相关性。

2. 数据增强 (Data Augmentation)

数据增强是在检索前对数据进行预处理，如去除不相关信息、消除歧义、更新数据等。通过合成新的数据，可以有效提高最终 RAG 系统的性能。

具体的数据优化方法有以下几种方式：

- **数据清洗**：这一步包括使用数据加载器 (Data Loader) 提取不同格式的数据，如 PDF、Word、Markdown 文件以及数据库和 API 等。
- **数据处理**：这一过程涉及数据格式处理、剔除不可识别内容、压缩和格式化数据等。
- **元数据提取**：提取文件名、时间、章节标题、图片 alt 文本等信息，这些元数据对于提高检索的准确性和效率非常关键。

2. 检索增强 (Retriever Enhancement)

检索过程在 RAG 系统中至关重要。检索内容的质量越高，越能激发大型语言模型 (LLMs) 的上下文学习能力以及其他生成器的效能。内容质量差，则模型产生幻觉的可能性越大。因此，关键在于如何有效提高检索过程的有效性。

1. 递归检索 (Recursive Retrieve)

递归检索是指进行多次搜索以获取更丰富、质量更高的内容。ReACT 使用思维链 (Chain-of-Thought, CoT) 技术对查询进行拆分，进行递归检索，提供更丰富的信息。RATP 利用蒙特卡罗树搜索 (Monte-Carlo Tree Search, MCTS) 进行多次模拟，识别出最优的检索内容。然后，这些内容被整合到一个模板中，发送给生成器进行最终生成。通过递归检索，查询被逐步分解，模型能够提供更全面的相关知识。

2. 块优化 (Chunk Optimization)

块优化技术通过调整块的大小来获得更好的检索结果。句子窗口检索是一种有效的方法，它通过获取小块文本并返回包含检索段落前后相关句子的窗口，从而增强检索效果。这种方法确保了目标句子前后的上下文都包含在内，提供了更全面的信息。

自动合并检索是 LlamaIndex 的另一种高级 RAG 方法，它将文档组织成树状结构，父节点包含

所有子节点的内容，如文章和段落、段落和句子之间的亲子关系。在检索过程中，精细搜索子节点，最终返回父节点，提供更丰富的信息。

句子窗口检索通过获取小段文本并返回包含检索段落上下文的句子窗口，确保对检索信息的全面理解。自动合并检索将文档组织成树状结构，通过先获取子节点，最终检索到包含子节点内容的父节点，从而提供更丰富的信息。为解决上下文信息不足的问题，RAPTOR 使用递归嵌入、文本块聚类 and 摘要技术，直到进一步聚类变得不可行，从而构建多级树结构。

3. 检索器微调 (Finetune Retriever)

作为 RAG 系统的核心组件，检索器在整个系统运行过程中起着关键作用。有效的嵌入模型能够在向量空间中聚类语义相似的内容，增强检索器为后续生成器提供有价值信息的能力，从而提高 RAG 系统的效率。因此，嵌入模型的性能（如 bge_embedding、bge_m3、cocktail、llm_embedder）对 RAG 系统的整体有效性至关重要。

对于已经具有良好表达能力的嵌入模型，我们仍然可以使用高质量的领域数据或任务相关数据对其进行微调，以提高其在特定领域或任务中的性能。例如，REPLUG 将语言模型视为黑盒，根据最终结果更新检索器模型。具体方法包括：

- **APICoder** 使用 Python 文件和 API 名称、签名、描述对检索器进行微调。
- **EDITSUM** 对检索器进行微调，以减少检索后摘要之间的编辑距离。
- **SYNCHROMESH** 在损失函数中添加 AST 的树距离，并使用目标相似度调优来微调检索器。
- **R-ConvED** 使用与生成器相同的数据对检索器进行微调。

4. 混合检索 (Hybrid Retrieve)

混合检索指的是同时使用多种检索方法或从多个不同来源提取信息。RAP-Gen 和 ReACC 同时使用密集检索器和稀疏检索器来提高检索质量。具体实现包括：

- **Rencos** 使用稀疏检索器在句法层面检索相似的代码片段，使用密集检索器在语义层面检索相似的代码片段。
- **BASHEXPLAINER** 首先使用密集检索器捕捉语义信息，然后使用稀疏检索器获取词汇信息。
- **RetDream** 首先使用文本检索，然后使用图像嵌入进行检索。

其他混合检索方法包括：

- **CRAG** 设计了检索评估器，评估检索文档与输入查询的相关性，根据不同的置信度水平触发三种检索操作：如果判断正确，直接使用检索结果进行知识提炼；如果错误，转而使用网络搜索；如果模糊，两者结合。

- **RAGAE** 在检索阶段引入了两种指标，DKS（密集知识相似度）和 RAC（检索器作为答案分类器），这些指标既考虑了答案的相关性，也考虑了底层知识的适用性。
- **UniMS-RAG** 引入了一种新的标记，称为“行动令牌”，它决定了从何处检索信息。

5. 重新排序（Re-ranking）

重新排序技术是指对检索到的内容重新排序，以达到更大的多样性和更好的结果。Re2G 采用了继传统检索器之后的重新排序模型，以减少将文本压缩成向量时信息丢失对检索质量的影响。AceCoder 使用选择器对检索到的程序重新排序，目的是减少冗余程序，获得多样化的检索结果。XRICL 在检索后使用基于蒸馏的范例重新排序器进行优化。

具体方法包括：

- **Re2G** 使用重排器模型来减少文本向量化导致的信息丢失对检索质量的影响。
- **AceCoder** 通过选择器对检索到的程序进行重新排序，以减少冗余并获取多样化的结果。
- **XRICL** 使用基于示例的重排器进行优化。
- **Rangan** 利用量化影响度量评估查询与参考之间的统计偏差，以评估数据子集的相似性并重新排列检索结果。
- **UDAPDR** 利用大语言模型生成合成查询，训练领域特定的重排器，并应用多教师知识蒸馏构建连贯的检索器。
- **LLM-R** 通过静态 LLM 进行文档排序和奖励模型训练，结合知识蒸馏，迭代优化检索器，每次训练周期逐步提高检索器性能。

6. 检索转换（Retrieval Transformation）

检索转换涉及对检索到的内容进行改写，以更好地激发生成器的潜力，从而提高输出质量。FILCO 有效地从检索到的文本块中过滤出无关内容，只保留精确的支撑信息，简化生成器的任务。FiD-Light 首先使用编码器将检索内容转换为向量，然后压缩，显著减少了延迟时间。RRR 在每轮中通过模板将当前查询与前 k 个文档整合，然后通过预训练的 LLM（如 GPT-3.5-Turbo）进行重构。

具体方法包括：

- **FILCO** 高效剔除检索文本中的多余内容，只保留关键支撑信息，以简化生成器的任务并促进准确答案预测。
- **FiD-Light** 使用编码器将检索内容转换为向量，然后压缩，显著减少延迟时间。

- **RRR** 通过模板将当前查询与前 k 个文档整合，并通过预训练的 LLM 进行重构。

7. 其他优化方法

除了上述优化方法，还有其他一些针对检索过程的优化技术。例如，元数据过滤（Meta-data Filtering）是一种利用元数据（如时间、目的等）筛选检索文档以获得更好结果的方法。

GENREAD 和 GRG 引入了一种新颖的方法，通过提示大语言模型生成文档来替代或改进检索过程，以响应给定的问题。

3. 生成器增强（Generator Enhancement）

在 RAG 系统中，生成器的质量往往决定了最终输出结果的质量。因此，生成器的能力决定了整个 RAG 系统效能的上限。

1. 提示工程（Prompt Engineering）

提示工程中的技术侧重于提高大语言模型（LLM）输出的质量，如提示压缩（Prompt Compression）、Stepback Prompt、Active Prompt、Chain of Thought Prompt（CoT）等。这些技术都适用于 RAG 系统中的 LLM 生成器。例如：

- **LLM-Lingua** 通过小型模型压缩查询的总体长度，以加速模型推理，减轻无关信息对模型的负面影响，缓解“迷失在中间”现象。
- **ReMoDiffuse** 通过使用 ChatGPT，将复杂描述分解为结构化文本脚本。
- **ASAP** 在提示中添加示例元组以获得更好的结果，示例元组包括输入代码、函数定义、分析结果及其关联注释。
- **CEDAR** 使用设计好的提示模板将代码演示、查询和自然语言指令组织到一个提示中。
- **XRICL** 利用 CoT 技术添加翻译对作为跨语言语义解析和推理的中间步骤。
- **Make An Audio** 能够使用其他模态作为输入，为后续过程提供更丰富的信息。

2. 解码调优（Decoding Tuning）

解码调优指的是在生成器处理过程中添加额外控制，可以通过调整超参数来增加多样性，限制输出词汇等方式实现。例如：

- **Interfix** 通过调节解码器的温度来平衡结果的多样性和质量。
- **SYNCHROMESH** 通过实现补全引擎来消除实现错误，从而限制了解码器的输出词汇表。
- **DeepICL** 根据额外的温度因素控制生成的随机性。

3. 生成器微调（Generator Finetuning）

生成器的微调可以增强模型在特定领域知识方面的精确性或更好地适应检索器。例如：

- **RETRO** 固定了检索器的参数，并使用生成器中的分块交叉注意机制将查询内容与检索器结合起来。
- **APICoder** 对生成器 CODEGEN-MONO350M 进行微调，将一个经过洗牌的新文件与 API 信息和代码块结合起来。
- **CAREt** 首先使用图像数据、音频数据和视频文本对编码器进行训练，然后以减少标题损失和概念检测损失为目标对解码器（生成器）进行微调，在此过程中，编码器和检索器被冻结。
- **Animation-a-Story** 使用图像数据优化视频生成器，然后对 LoRA 适配器进行微调，以捕获给定角色的外观细节。
- **Ret-Dream** 用渲染的图像微调 LoRA 适配器。

4、结果增强（Result Enhancement）

在很多场景下，RAG 的最终结果可能达不到预期效果，一些结果增强技术可以帮助缓解这一问题。输出重写是指在某些场景中，对生成器产生的内容进行重写，以满足下游任务的需求。

例如：

- **SARGAM** 通过 Levenshtein Transformer 对删除分类器、占位分类器和插入分类器进行分类，修改代码相关任务中生成的结果，以更好地适应实际的代码上下文。
- **Ring** 通过重新排序得到多样性结果，候选基于生成器生成的每个 token 日志概率的平均值。
- **CBR-KBQA** 通过将生成的关系与知识图中查询实体的局部邻域中的关系对齐来修正结果

Ring 通过重新排序得到多样性结果，候选基于生成器生成的每个 token 日志概率的平均值。

CBR-KBQA 通过将生成的关系与知识图中查询实体的局部邻域中的关系对齐来修正结果。

5. RAG 流程增强（RAG Pipeline Enhancement）

1. 自适应检索（Adaptive Retrieval）

实际经验表明，检索并不总是有利于最终生成的结果。当模型本身的参数化知识足以回答相关问题时，过度检索会造成资源浪费，并可能增加模型的混乱。因此，在本章中，我们将讨论确定是否检索的两种方法：基于规则的方法和基于模型的方法。

- **基于规则（Rule-based）：**
 - **FLARE** 在生成过程中通过概率主动决定是否搜索以及何时搜索。

- **Efficiency-KNNLM** 将 KNN-LM 和 NPM 的生成概率与超参数 λ 相结合，以确定生成和检索的比例。
- **Malen 等人** 在生成答案前对问题进行统计分析，允许模型直接回答高频问题，对低频问题引入 RAG。
- **Jiang 等人** 研究了模型不确定性、输入不确定性和输入统计量，综合评估模型的置信水平，决定是否检索。
- **Kandpal 等人** 通过研究训练数据集中相关文档的数量与模型掌握相关知识的程度之间的关系，帮助确定是否需要检索。

○ **基于模型 (Model-based):**

- **Self-RAG** 使用经过训练的生成器根据不同指令判断是否执行检索，并通过 Self-Reflection 评估检索文本的相关性和支持程度。最终，根据 Critique 评估最终输出结果的质量。
- **Ren 等人** 使用“判断提示”来判断大模型是否能够回答相关问题以及答案是否正确，从而帮助确定检索的必要性。
- **SKR** 利用大语言模型自身提前判断是否能回答问题，如果能回答，则不进行检索。

2. 迭代 RAG (Iterative RAG)

迭代 RAG 通过反复循环检索和生成阶段，逐步精炼结果，而非单轮处理。

- **RepoCoder** 采用迭代检索生成管道，更好地利用分散在不同文件中的有用信息。在每次迭代期间使用先前生成的代码增强检索查询，获得更好的结果。
- **ITER-RETGEN** 以迭代方式协同检索和生成，生成器当前的输出反映其缺乏的知识，检索器检索缺失的信息作为下一轮的上下文信息，提高下一轮生成内容的质量。

RAG 应用与分类:

RAG for Text					
Question Answering	Human-Machine Conversation	Neural Machine Translation	Summarization	Others	
REALM ^{‡§} TKEGEN [§] TOG [‡] SKR ^{§¶} Self-RAG ^{§¶} RIAG [‡] FID ^{‡§} RETRO [§] NPM ^{‡§}	CREA-ICL ^{‡‡} BlenderBot ^{‡§} CEG [‡] Internet-Augmented-DG ^{‡§} ConceptFlow ^{‡§} Skeleton-to-Response ^{‡§}	NMT-with-Monolingual-TM ^{‡‡§} TRIME ^{‡§} KNN-MT ^{‡§} COG [‡]	RAMKG ^{‡§} RPRR [‡] RIGHT ^{‡§} Unlimiformer [§]	CONCRETE ^{‡§} Atlas ^{‡§} KG-BART ^{‡§} R-GQA ^{‡§}	
RAG for Code					
Code Generation	Code Summarization	Code Completion	Automatic Program Repair	Text-to-SQL and Code-based Semantic Parsing	Others
SKCODER [§] RRGCode [‡] ARKS ^{‡¶} RECODE KNN-TRANX Toolcoder [§]	RACE [‡] BASHEXPLAINER [‡] READSUM Rencos [‡] CoRec [‡] Tram [§] EDITSUM [‡]	ReACC ^{‡‡} RepoCoder ^{‡§¶} De-Hallucinator [¶] REPOFUSE [§] RepoFusion [§] EDITAS [§]	RING CEDAR [§] RAP-Gen ^{‡§} InferFix [§] SARGAM [§] RTLFixer ^{‡§}	XRICL ^{‡§} SYNCHROMESH ^{‡§} RESDSL [§] REFSQL ^{‡§} CodeCL [§] MURRE [¶]	De-fine [‡] Code4UIE [§] E&V [§] StackSpotAI ^{‡§} InputBlaster [¶]
RAG for Knowledge					RAG for 3D
Knowledge Base QA	Knowledge-augmented Open-domain QA		Table for QA	Others	Text-to-3D
CBR-KBQA ^{‡§} TIARA ^{‡‡§} Keqing ^{‡‡§} RNG-KBQA [‡] ReTraCk [§] SKP ^{‡§§}	UniK-QA ^{‡‡} KG-FID [‡] GRAPE [‡] SKURG ^{‡‡} KnowledGPT [‡] EFSUM [§]		EfficientQA [‡] CORE [§] Convinse ^{‡‡} RINK ^{‡§} T-RAG ^{‡§} StructGPT [‡]	GRetriever [§] SURGE [§] K-LaMP [§] RHO	ReMoDiffuse ^{‡‡} AMD [‡]
RAG for Image			RAG for Video		
Image Generation	Image Captioning	Others	Video Captioning	Video QA & Dialogue	Others
RetrieveGAN [‡] IC-GAN [§] Re-Imagen [§] RDM [§] Retrieve&Fuse [§] KNN-Diffusion	MA REVEAL [‡] SMALLCAP [‡] CRSR [‡] RA-Transformer	PICa Maira [‡] KIF [‡] RA-VQA [‡]	KaVD ^{‡§} R-ConvED ^{‡§} CARE [§] EgoInstructor ^{‡‡§}	MA-DRNN ^{‡‡} R2A [‡] Tvqa ^{‡§} VGNMN [‡]	VidIL ^{‡‡} RAG-Driver [‡] Animate-A-Story ^{‡§}
RAG for Science			RAG for Audio		
Drug Discovery	Biomedical Informatics Enhancement		Math Applications	Audio Generation	Audio Captioning
RetMol ^{‡§} PromptDiff ^{‡‡} PoET [‡] Chat-Orthopedist ^{‡§} BIOREADER [‡] MedWriter [‡] QARAG ^{‡‡} LeanDojo [‡] RAG-for-math-QA ^{‡‡}				Re-AudioLDM [§] Make-An-Audio ^{‡§}	RECAP ^{‡§}
Query-based Latent-based Logit-based Speculative Query+Latent Latent+Logit ‡ Input ‡ Retriever § Generator Output ¶ Pipeline					

这一块细节可以参考论文原文。[Retrieval-Augmented Generation for AI-Generated Content: A Survey](#)

RAG 评估与基准

评估检索增强生成（RAG）系统时面临的挑战，并将其分为三个主要部分：检索（Retrieval）、生成（Generation）和整个 RAG 系统（作为整体）。

检索组件的挑战

- **动态和广泛的知识库**：评估检索组件时，需要处理的挑战之一是潜在知识库的动态性和广泛性，这要求评估指标能够有效地衡量检索文档的精确度、召回率和相关性。
- **时间敏感性**：信息的相关性和准确性会随时间变化，这增加了评估过程的复杂性。
- **信息源的多样性**：评估检索组件时还需考虑信息源的多样性，以及检索到误导性或低质量信息的可能性。

生成组件的挑战

- **忠实度和准确性**：生成组件的评估重点在于生成内容对输入数据的忠实度和准确性，这不仅涉及事实正确性，还包括对原始查询的相关性和生成文本的连贯性。
- **主观性**：某些任务（如创意内容生成或开放式问题回答）的主观性增加了评估的复杂性，因为它们引入了关于“正确”或“高质量”响应的变异性。

RAG 系统作为整体的挑战

- **检索与生成的相互作用**：整个 RAG 系统的评估引入了额外的复杂性，因为检索和生成

组件之间的相互作用意味着不能仅通过独立评估每个组件来完全理解整个系统的性能。

- **实际考虑：**评估系统的整体有效性和可用性时，还需要考虑响应延迟、对错误信息的鲁棒性以及处理模糊或复杂查询的能力。